

Books

→ Ian Sommerville, Software engineering, Addison-Wesley (9th edition).

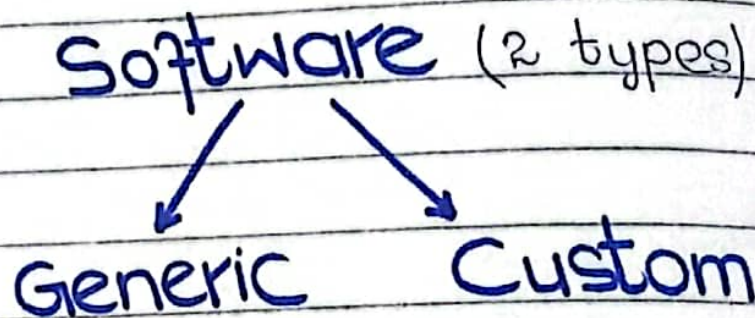
→ Software engineering: A practitioner's approach, Roger Pressman, McGraw Hill, 7th edition.

Software Eng.

Introduction to software engineering:

⇒ Software:

Software is the programs and documents that help a computer work, like instructions and guides.



• Generic software:

It is a regular program that anyone can use, like a basic word processor or a calculator app.

• Custom software:

It is made just for one person or business. It's designed to fit

e their specific needs, like a special app for tracking sales.

⇒ Software engineering:

Software engineering is a field that focuses on all parts of making software. This starts with figuring out what the software needs to do, and it continues through building it and fixing it after it's being used.

→ It also focuses on project management and the development of tools.

⇒ Difference:

CS	SE
→ concerned with <u>theory</u> and <u>fundamentals</u> .	→ concerned with <u>practicalities of developing and delivering</u> useful software.

→ CS theories are still insufficient to act as a solid foundation.

⇒ Difference:

SE

Software engineering is the part of this process that focuses specifically on creating and maintaining software applications.

System engineering

System engineering is about designing and managing different parts of a system so they work well together.

⇒ Software process:

It is a collection of activities, action and task that are needed to be performed in order to create one product.

Activities:

Specification:

what the system should do.

Development:

production of the software system.

Validation:

checking → according to customer demand.

Evolution:

Changing in response to changing demands.

⇒ Software process models:

• Waterfall

Sequential development.

• Incremental development:

Developing and delivering the software in pieces.

• Component-Based:

Using existing components to build software.

⇒ Costs of software engineering:

Development: 60% of total costs.

Testing: 40%

⇒ For custom software, evolution often exceeds development costs.

⇒ CASE (Computer-aided SE) :

Tools that support software development activities.

Upper case:

Helps with requirements and designs.

Lower CASE:

Supports coding, debugging and testing.

⇒ Attributes of good software:

• Maintainability:

Software should evolve to meet new needs.

• Dependability:

Reliability, security and safety.

• Efficiency:

Responsiveness, good use of resources.

• Acceptability:

Software must be usable and compatible.

⇒ Challenges in software engineering:

• Heterogeneity:

Software must operate across different systems and devices.

• Business and social changes:

Software must adapt to rapid changes.

• Security and trust:

As software becomes central to our lives, ensuring its security is critical.

=> Reasons for software failures:

• Increasing demands:

Systems are becoming more complex and need fast delivery.

• Lack of SE techniques:

Without proper software engineering methods, software becomes expensive and unreliable.

=> Application types:

. Stand-alone applications:

Run on local PCs (e.g. word processor).

. Interactive transaction based systems:

Run on remote computers, accessed via web (e.g. e-commerce).

. Embedded control systems:

Control hardware devices (e.g. microwave oven).

. Batch processing systems:

Process large amounts of data (e.g. billing systems).

. Entertainment systems:

Games and personal software.

. Systems for modelling and simulation:

Used by scientists to simulate real world scenarios.

. Data collection systems:

Use sensors to gather data (e.g. temperature sensors).

. Systems of systems:

Composed of several integrated software systems.

⇒ Software engineering fundamental principles:

Systems should be developed using well-managed processes.

Dependability and performance are crucial.

Reusing existing software when possible is beneficial.

⇒ Software engineering and the web.

Access functionality via the web.

⇒ Cloud computing (SaaS):

Applications run remotely, and users pay based on usage.

Incremental development:

Web-based systems are developed and delivered incrementally.

⇒ Software engineering ethics:

Software must act responsibly and ethically.

• Confidentiality:

Respect clients' confidentiality.

• Competence:

Work within your expertise.

Intellectual property:

Respect patents and copyrights.

Avoid misuse:

Don't misuse computers e.g; by spreading viruses.

